Sorting Performance Comparison Web App: Music Playlist Manager

Cheolwon Park (Student ID: 2024270616)
Korea University
scott9020@korea.ac.kr

1 Web Application and Algorithm Implementation

A music playlist manager web app was built using Flask (Python backend) and HTML/CSS/JavaScript (frontend). The application generates random playlists of N songs with randomized titles, artists, durations (120-420s), and play counts (0-100,000). Six sorting algorithms were implemented, each tracking **comparisons**, **swap/moves**, and **execution time**:

Algorithm	Worst Case	Type	Implementation
Selection	$O(n^2)$	In-place	Find minimum in unsorted portion, swap to front
Insertion	$O(n^2)$	In-place	Shift elements right to insert current item in order
Merge	O(n log n)	Not in-place	Recursive divide-and-conquer with merge step
Bubble	$O(n^2)$	In-place	Adjacent swaps with early termination optimization
Quick	$O(n^2)$	In-place	Last-element pivot, Lomuto partition scheme
Heap	O(n log n)	In-place	Max-heap construction, repeated extraction

Users can sort by title, artist, duration, or play count. The "Compare All" runs all six algorithms on the same playlist and displays a comparison table highlighting the best values in green.

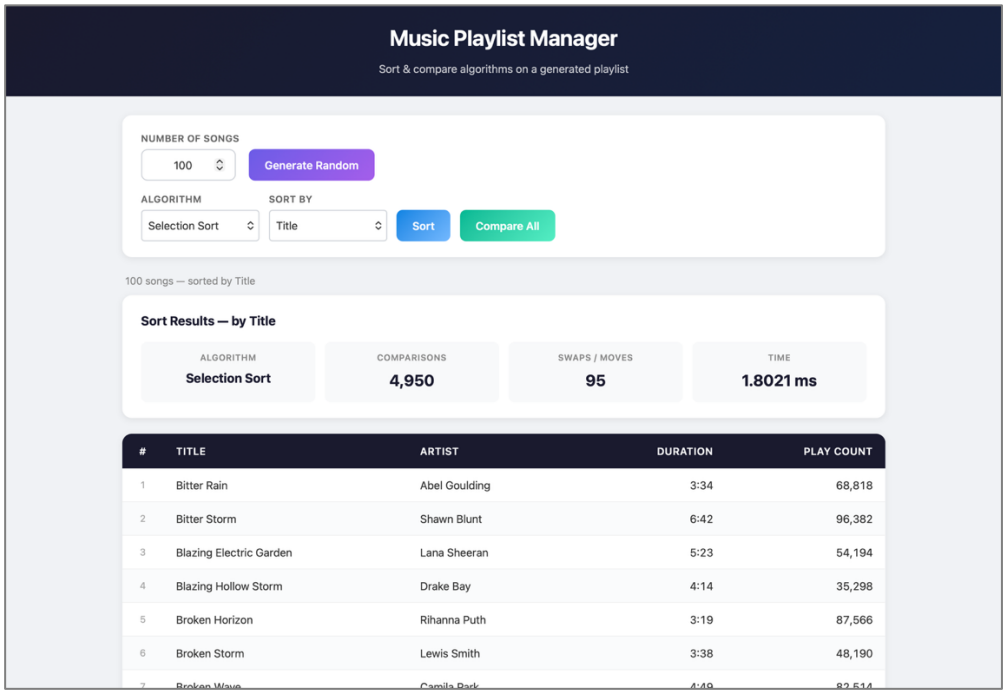

Figure 1. Playlist manager showing selection sort results for 100 songs sorted by Title

2 Sorting Performance Comparison (N = 100, 1000, 5000)

Each algorithm was tested on randomly generated playlists sorted by title (string comparison). All tests used `time.perf_counter()` for precise timing.

Algorithm	Complexity	Comparisons			Swaps / Moves			Time (ms)		
		100	1,000	5,000	100	1,000	5,000	100	1,000	5,000
Selection	$O(n^2)$	4,950	499,500	12,497,500	95	991	4,994	2.05	101.90	2108.93
Insertion	$O(n^2)$	2,452	247,151	6,313,596	2,358	246,159	6,308,609	1.12	46.05	1223.36
Bubble	$O(n^2)$	4,884	497,015	12,491,059	2,358	246,159	6,308,609	2.31	106.35	2790.25
Merge	O(n log n)	546	8,723	55,183	546	8,723	55,183	0.52	2.93	17.42
Quick	O(n log n)	632	10,690	71,471	413	5,631	38,502	0.23	1.78	12.26
Heap	O(n log n)	1,026	16,865	107,612	582	9,098	57,052	0.54	4.25	28.01

Table 1. Performance comparison across playlist sizes (sorted by title)

3 Analysis

3.1 Which algorithm was fastest? Why?

Quick Sort was the fastest at every scale: 0.23 ms (N=100), 1.78 ms (N=1,000), and 12.26 ms (N=5,000). All three O(n log n) algorithms — Quick, Merge, and Heap — stayed under 30 ms at N=5,000, while Bubble Sort took 2,790 ms (at least 93× slower). $O(n^2)$ algorithms like Selection and Bubble Sort compare every pair of elements using nested loops, resulting in $\sim n^2/2$ comparisons (e.g. 12.5M at N=5,000). O(n log n) algorithms avoid this by dividing the problem: Quick Sort partitions into subproblems, Merge Sort splits and merges, each needing only $\sim n \cdot \log_2 n$ comparisons (e.g. 55–71K at N=5,000). Among the O(n log n) group, Quick Sort wins because it partitions in-place with sequential memory access, giving it better cache locality than Merge Sort (which allocates new arrays) or Heap Sort (which jumps between parent/child indices).

3.2 $O(n^2)$ vs. O(n log n) scaling

Increasing the input 50× (100 → 5,000) caused Selection Sort's comparisons to grow 2,525× (4,950 → 12.5M), matching the theoretical $50^2 = 2{,}500$. Merge Sort grew only 100× (546 → 55K). In execution time, Bubble Sort went from 2 ms to 2,790 ms, while Quick Sort went from 0.23 ms to 12 ms. The gap between $O(n^2)$ and O(n log n) widens rapidly and becomes impractical beyond a few thousand elements.

3.3 When are $O(n^2)$ algorithms acceptable?

- **Small n (< 50):** All algorithms finished under 2.5 ms at N=100. Simpler algorithms have lower constant factors and no recursion overhead.
- **Nearly sorted data:** Insertion Sort achieves O(n) best-case, ideal for maintaining an already-ordered list with few additions.
- **Memory constraints:** Selection/Insertion Sort use O(1) extra space, unlike Merge Sort's O(n). Critical on embedded systems.
- **As part of hybrids:** Timsort uses Insertion Sort for small subarrays ($n \leq 32$) within Merge Sort, combining low overhead with scalability.